safeguard its APIs, and how OAuth fits into the picture as a framework for delegated access.

10.5 Summary

- Effective policies are more than just one-off rules; they are design artifacts that must remain clear, consistent, and aligned with business needs over time.
- Clarity and consistency help prevent conflicts. At ACME, policies defined by project membership were more predictable than *ad hoc* rules based on organizational roles.
- Least privilege prevents overexposure. A broad “all employees” permit gave interns access to sensitive forecasts, but tightening policies to only include project members closed the gap.
- Separating policy from data makes rules more stable. Policies that check durable roles or teams are better at surviving organizational change than those that use hard-coded department names or other data that is frequently updated.
- Granularity matters. Fine-grained actions like view, edit, and share offer precision, while grouping them into higher-level roles in the UI keeps things simple.
- Policies tied to stable entities like teams or projects are resistant to change. An employee’s promotion might violate a title-based policy, for example, but wouldn’t affect one based on the employee’s team.
- Policy repositories and policy stores serve different functions. Repos are used for authoring, history, and review; stores are for runtime enforcement with signed, versioned bundles.
- Schemas ensure consistency. By validating policies against schemas, ACME avoided mismatched action names and discovered missing attributes before deployment.
- Versioning and review promote accountability. Pull requests record the reasoning for each policy change and enable reviewers to question scope and intent.
- Operational practices ensure policies are auditable. Decision logs, explainability, and rollback features provide developers and operators